

EXPECTED OUTCOMES

Console program (Java) and frontend website (HTML, CSS, JavaScript)

Project Title	Dictionary-Based Error Detection and Suggestions in Text Processing System	
Group No.	16	
Team Members	Roll Number	Name
1	2520030237	Hansika Reddy Gunapati
2	2520030048	Teja Hasini Reddy
3	2520030064	Anthony Nissy

1. Summary of Expected Outcomes

#	Outcome	What the project delivers
1	Error detection	Every word missing from the 370,105-word dictionary is flagged as a potential error with its line and word position (Trie lookup, $O(L)$ per word).
2	Ranked suggestions	Each error gets the top 3 corrections ranked by Damerau-Levenshtein edit distance, with Soundex and common-word ranking.
3	Error location (KMP)	Each error is located with exact [start ... end] character indices, line, column and word number.
4	Multi-pattern scan (Aho-Corasick)	All error patterns are found together in a single pass over the text, $O(n + z)$.
5	Conflict-free assignment (Max Flow)	Edmonds-Karp and Dinic assign a different correction to every error, so no two errors take the same word.
6	Measured accuracy	On 70 labelled test pairs: precision 100%, recall 90%, F1 94.7%, top-1 90%, top-3 90%.
7	Algorithm comparison	A comparison matrix shows time and space complexity, methodology and best use of each algorithm.

2. Console Program Outputs

A. Spell check with suggestions Java

Expected outcome: all 3 misspelled words are detected, each with 3 ranked suggestions, and the corrected sentence is produced.

```
$ java -cp bin spellcheck.Main "Helo hapy borthday"
Dictionary words loaded: 370,105
Input Type : Short Sentence
Selected Strategy: Simple Dictionary Lookup + Edit Distance

Detected Potential Errors: 3
Line 1, Word 1: Helo
Line 1, Word 2: hapy
Line 1, Word 3: borthday

Suggestions (best first, with edit distance):
helo -> hello (1), help (1), helio (1)
hapy -> happy (1), heapy (1), haply (1)
borthday -> birthday (1), birthdays (2), bothway (2)

Corrected Output: Hello happy birthday
Total Tokens : 3   Errors Found : 3   Suggestions Made : 9
```

B. KMP single-pattern search Menu 3

Expected outcome: the text is spell-corrected first, then KMP finds the exact position of the pattern.

```
$ Menu 3 - text: what ie your name | pattern: is your
Original Text      : what ie your name
Corrected Text     : what is your name
Text corrections   : ie -> is
Original Pattern   : is your
LPS table          : [0, 0, 0, 0, 0, 0, 0]
Match Result       : FOUND (1 occurrence(s))
Match 1 at index 5 (line 1, column 6) -> "is your"
```

C. Aho-Corasick multi-pattern search Menu 4

Expected outcome: all patterns are found in one pass, and a misspelled pattern is corrected before searching.

```
$ Menu 4 - patterns: is your, recieve
Original Text      : what ie your name and I recieve a message
Corrected Text     : what is your name and I receive a message
Pattern corrections: recieve -> receive
Number of Patterns : 2   (Trie nodes: 15)
is your : 1 match(es) at 5 (line 1, column 6)
receive : 1 match(es) at 24 (line 1, column 25)
Patterns found     : 2 of 2
Total matches      : 2
```

D. Max Flow correction assignment Menu 7

Expected outcome: each error receives a different correction, and max-flow equals min-cut.

```
$ Menu 7 - sentence: I recieve a mesage
Network: 2 error word(s), 6 candidate correction(s)
Assignment (found by Edmonds-Karp):
  recieve -> receive
  mesage -> message
Edmonds-Karp : max-flow = 2, augmenting paths = 2
Dinic        : max-flow = 2, phases = 1
Min-cut capacity : 2 (equals max-flow: true)
```

E. Evaluation on labelled test data Menu 5

Expected outcome: accuracy is measured on corpus/test_data.csv, not guessed.

```
$ Menu 5 - test file: corpus/test_data.csv
Test pairs : 70 (real errors: 60, correct words: 10)
DETECTION  TP=54 FP=0 FN=6 TN=10
Precision  : 100.0%
Recall     : 90.0%
F1-score   : 94.7%
CORRECTION (over 60 real errors)
Top-1 correction accuracy : 90.0% (54/60)
Top-3 suggestion recall   : 90.0% (54/60)
```

The 6 undetected cases are misspellings that happen to be valid dictionary words (for example "calender"). Dictionary lookup cannot flag these.

3. Frontend Website Outputs

A. Spell Checker tab Trie + Edit Distance

Expected outcome: typing "Helo hapy borthday" highlights all 3 errors and shows total words, errors detected and text accuracy.

The screenshot displays the 'Dictionary-Based Spell Checker (Trie + Edit Distance)' web application. At the top, there are 'Quick Test Presets' buttons: 'Preset 1: Demo ("wat is yoe nama...")', 'Preset 2: Project Typos', 'Preset 3: Frequent Errors', and 'Preset 4: Flow Collisions'. Below these is a text input field containing 'Helo hapy borthday'. To the right of the input field is a button labeled 'Auto-Apply All Top-1 Corrections'. Below the input field is a section titled 'Dictionary-Based Spell Checker (Trie + Edit Distance)' with a descriptive paragraph. Underneath this is a statistics section with four cards: 'TOTAL WORDS' showing '3', 'ERRORS DETECTED' showing '3' in red, 'TEXT ACCURACY' showing '0.0%', and 'TRIE + DP TIME' showing '284.00 ms'. At the bottom is a 'Document Highlighted Preview' section showing the input text 'Helo hapy borthday' with the words 'hapy' and 'borthday' highlighted in red.

Figure 1: Spell Checker input, statistics and highlighted preview

Expected outcome: each error shows its index and top 3 suggestions with edit distance, Soundex and common-word tags; one click replaces the word.

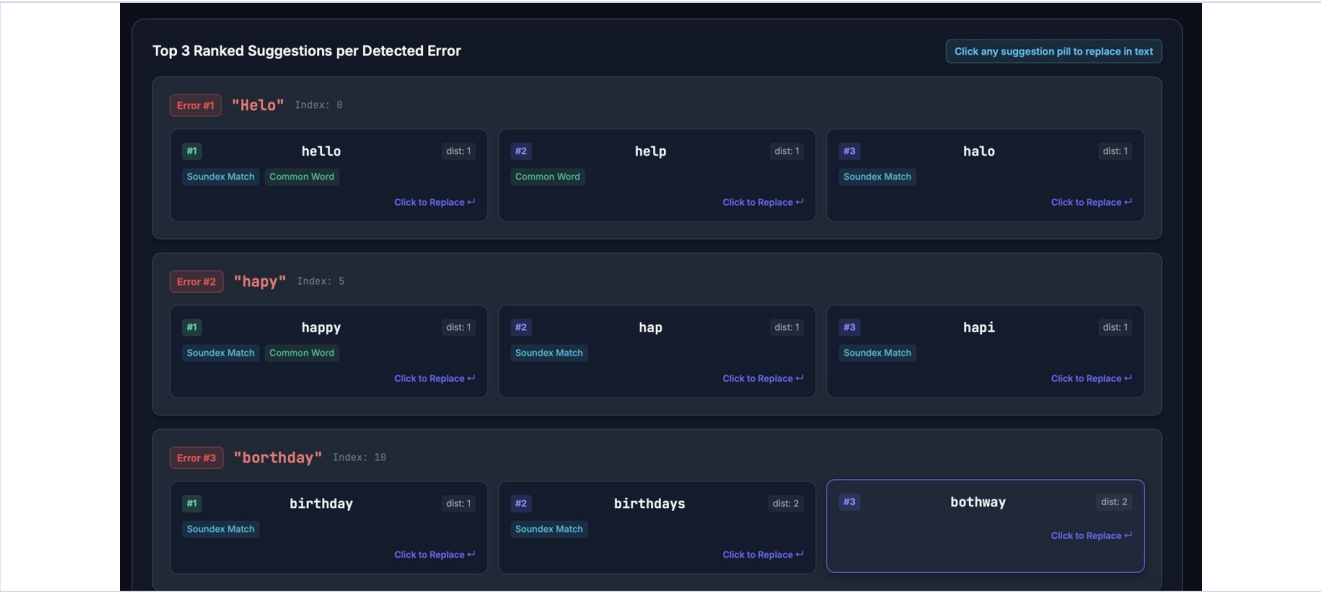


Figure 2: Top 3 ranked suggestions per detected error

B. KMP tab Error & Index Locator

Expected outcome: errors are found automatically, each with its [start ... end] range, line, column, word number and top 3 suggestions.

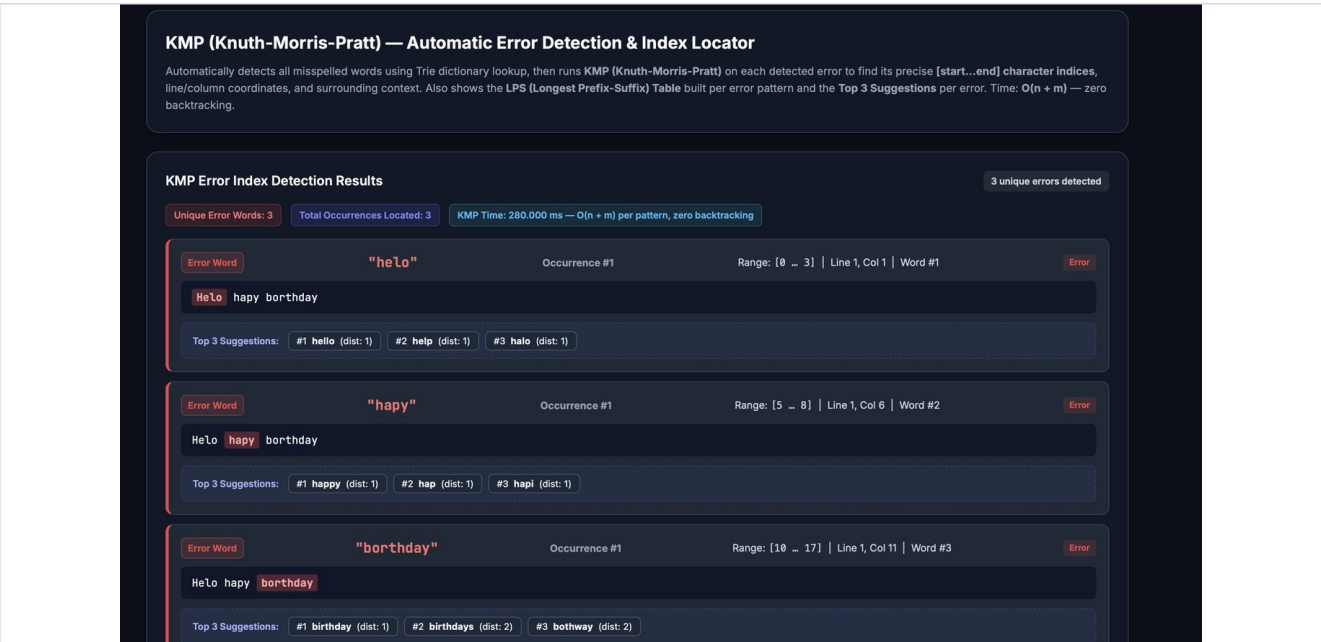


Figure 3: KMP error index detection results

C. Aho-Corasick tab Multi-Pattern Scanner

Expected outcome: all detected error words are loaded into one automaton and listed in a single table with index range and suggestions.

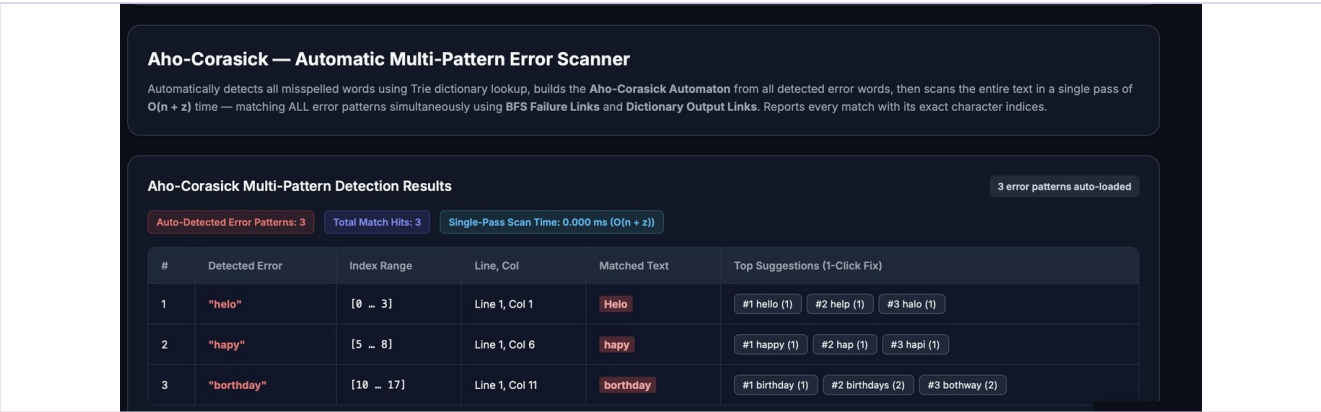


Figure 4: Aho-Corasick multi-pattern detection results

D. Max Flow tab Bipartite Suggestion Matcher

Expected outcome: errors "recieve" and "mesage" are matched to "receive" and "message", with every used edge saturated (1/1).

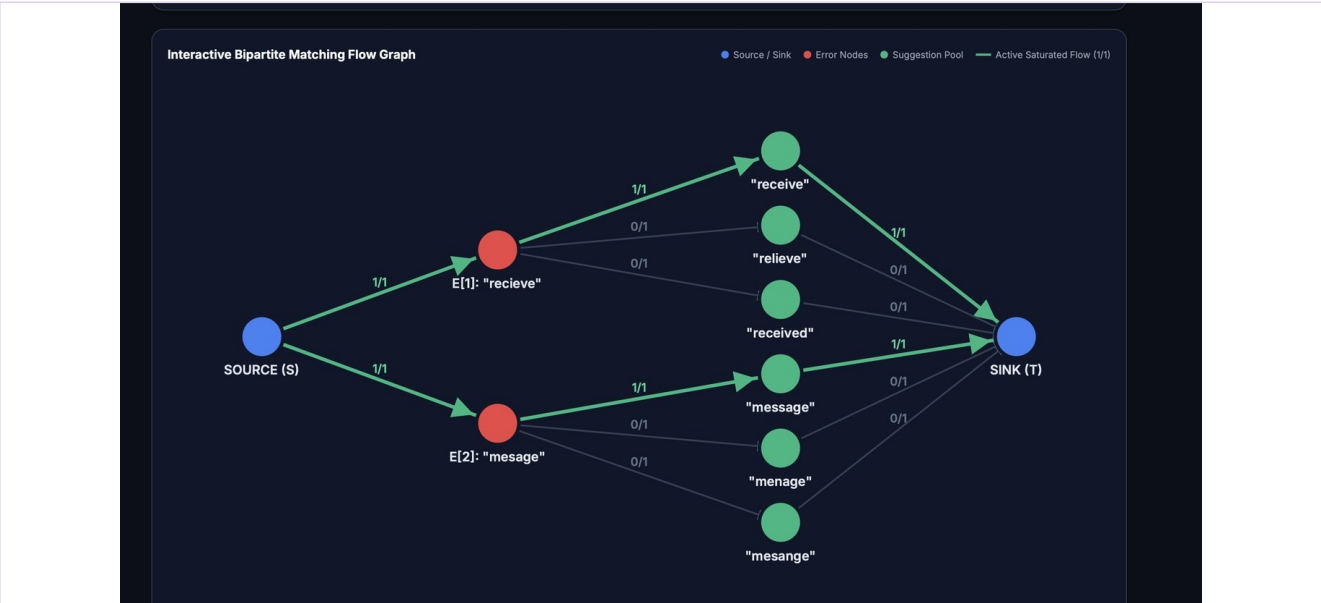


Figure 5: Interactive bipartite matching flow graph

Expected outcome: Edmonds-Karp finds 2 augmenting paths, reaches max flow 2, and stops when the sink is unreachable.

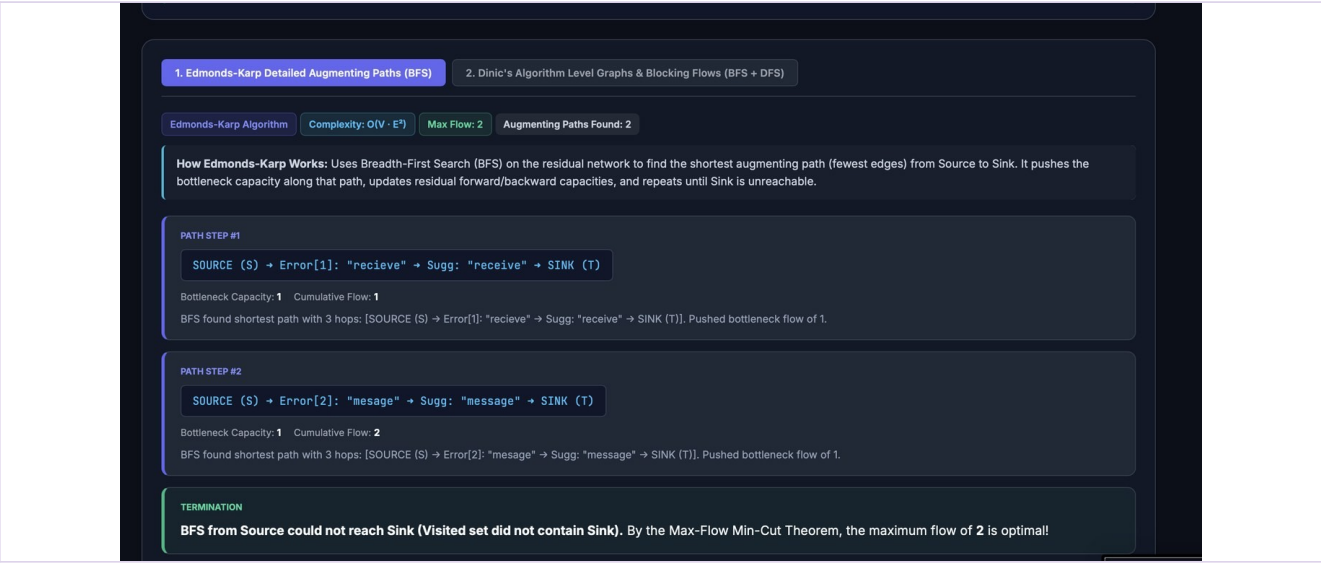


Figure 6: Edmonds-Karp step-by-step augmenting paths

E. Comparison tab Algorithm Comparison Matrix

Expected outcome: the matrix compares each algorithm on time and space complexity, methodology, output format, accuracy and real-world use.

Detailed Theoretical Comparison Matrix						
ALGORITHM & SYLLABUS CO	TIME COMPLEXITY	SPACE COMPLEXITY	METHODOLOGY	OUTPUT FORMAT	ACCURACY & RELIABILITY	BEST REAL-WORLD USE CASE
Trie + Damerau-Levenshtein CO2 (Trie) + CO3 (DP Edit Distance)	Build: $O(W \cdot L_{avg})$ [build Trie] Search: $O(L)$ per word lookup	$O(W \cdot L \cdot Alphabet_Size)$	Builds prefix tree where nodes represent characters. Traverses letter-by-letter in $O(L)$. If word is missing, computes dynamic programming edit distance table with length pruning and Soundex tie-break.	Token classification (Valid vs Potential Error), Top 3 ranked suggestions with edit distances (1-2) and phonetic codes.	94.7% F1 on labelled test sets. Detects any word missing from dictionary. Misses real-word typos (e.g. 'calender' vs 'calendar').	General-purpose interactive spell checkers (word processors, IDE code linters, search engine queries).
KMP (Knuth-Morris-Pratt) CO2 (String Algorithms - Primary)	Build: $O(m)$ [LPS Table] Search: $O(n)$ [Single pass search]	$O(m)$ auxiliary for LPS array	Precomputes Longest Prefix-Suffix (LPS) table on search pattern. Scans text with index 'i' that NEVER moves backwards. On mismatch, uses LPS to shift pattern. Extended here to report exact [start...end] character offsets, line/col numbers, and top 3 replacements.	Precise character range indices [start...end], line/column offsets, token position, surrounding context snippet, and error confirmation.	100% exact match for target error pattern. Cannot discover unknown typos independently; requires specified type pattern.	Finding and replacing targeted known errors, compiler error token locator, DNA nucleotide motif search, log file monitoring.
Aho-Corasick Automaton CO2 (String Algorithms - Multi-Pattern)	Build: $O(M \cdot Alphabet_Size)$ [Trie + BFS] Search: $O(n + z)$ [n = text, z = matches]	$O(M \cdot Alphabet_Size)$ for automaton nodes	Constructs a Trie of ALL search patterns simultaneously, then performs BFS to connect failure links (longest suffix transitions) and dictionary output links. Scans document once in $O(n + z)$ time.	Simultaneous detection of all known error patterns across the entire text with start/end indices and dictionary keyword flags.	100% exact multi-pattern discovery. Highly efficient for finding hundreds of known bad words / profanity / known misspelling databases.	Virus signature scanning, profanity/hate-speech filters, bulk multi-pattern replacement, network intrusion detection (Snort).

Figure 7: Detailed theoretical comparison matrix

4. Overall Result

Both versions detect misspelled words with a Trie, suggest corrections with Damerau-Levenshtein distance and Soundex, locate errors with KMP and Aho-Corasick, and use Max Flow to give every error a different correction. The console program reports measured accuracy on labelled data (F1 94.7%), and the frontend website shows the same algorithms step by step.